

Bitácora de Depuración — Práctica 1: El Asistente Frankenstein

Síntoma 1: Memoria corrupta al recargar

1. ¿Qué código generado por la IA estaba mal?

Código original:

```
function guardarMemoria() {  
  // Persistimos el contexto para la próxima sesión  
  localStorage.setItem(CLAVE_MEMORIA, historial);  
}
```

historial es un arreglo de objetos, pero localStorage solo almacena texto. La conversión implícita produce [object Object], que no es JSON válido. Al recargar, JSON.parse(guardado) genera un SyntaxError y detiene la restauración.

2. ¿Cómo usar las DevTools para detectar el error?

1. Abrir la aplicación original y enviar mensajes.
2. Abrir DevTools con F12 y entrar en Application > Local Storage.
3. Seleccionar el origen y buscar la clave memoria_llm.
4. Observar que Value contiene [object Object] en vez de objetos JSON con rol y texto.
5. Recargar y comprobar en Console el error de JSON.parse().

Figura 1. Application > Local Storage: memoria_llm contiene [object Object].



3. ¿Cuál fue su intervención como desarrolladores para solucionarlo y por qué funciona?

```
function guardarMemoria() {  
  localStorage.setItem(CLAVE_MEMORIA, JSON.stringify(historial));  
}
```

La carga utiliza:

```
historial = JSON.parse(guardado);
```

JSON.stringify() serializa el arreglo conservando rol y texto. JSON.parse() reconstruye la estructura al recargar, evitando la corrupción y permitiendo recorrer nuevamente el historial.

Síntoma 2: Comportamiento errático asíncrono

1. ¿Qué código generado por la IA estaba mal?

Endpoint incorrecto:

```
const ENDPOINT = "https://jsonplaceholder.typicode.com/postss";
```

Bloqueo del hilo principal:

```
function precalentar(ms) {  
  const fin = Date.now() + ms;  
  while (Date.now() < fin) { /* warm-up */ }  
}
```

Condición de carrera:

```
precalentar(700);  
consultarModelo();  
  
setTimeout(() => {  
  pintarMensaje("ia", respuestaPendiente);  
  estado.textContent = "✓ Respuesta recibida (fetch OK)";  
}, 600);
```

```
let respuestaPendiente;

async function consultarModelo() {
  try {
    const indice = historial.length + 1;
    const r = await fetch(ENDPOINT + "/" + indice);
    const datos = await r.json();
    respuestaPendiente = "🚗 " + datos.title;
  } catch (e) {
    // La API de prueba es muy estable; esto casi nunca pasa.
  }
}
```

El ciclo while congela la interfaz. El temporizador no espera el fetch y puede leer una respuesta indefinida, anterior o sobrescrita. Además, /postss devuelve un estado HTTP incorrecto y fetch no rechaza automáticamente una promesa por un 404.

2. ¿Cómo usar las DevTools para detectar el error?

1. En Sources, abrir el archivo original.
2. Colocar breakpoints en consultarModelo();, en la asignación a respuestaPendiente y en pintarMensaje("ia", respuestaPendiente);.
3. Usar Step Into para entrar en consultarModelo() y Step Over para avanzar hasta await fetch(...).
4. Observar que consultarModelo() se suspende y el callback del temporizador continúa en otro turno del event loop.
5. En el breakpoint del setTimeout, inspeccionar respuestaPendiente en Scope o Console.
6. En Call Stack, observar el callback del temporizador sin consultarModelo() en la misma pila.

CAPTURA PENDIENTE: Sources con breakpoint activo y panel Call Stack visible.

En Network, activar Fetch/XHR, enviar un mensaje y seleccionar la petición /postss/. Revisar Status, Time y Response. La versión defectuosa normalmente muestra 404 y no devuelve un title válido. Después de corregir, la URL /posts/{id} debe responder 200 con JSON y la propiedad title.

Figura 2. Network: fetch con estado 404 y tiempos de 54–55 ms. Falta una toma con Response abierto.



3. ¿Cuál fue su intervención como desarrolladores para solucionarlo y por qué funciona?

```
const ENDPOINT = "https://jsonplaceholder.typicode.com/posts";

async function consultarModelo() {
  const indice = historial.length + 1;
  const respuestaHttp = await fetch(`${ENDPOINT}/${indice}`);

  if (!respuestaHttp.ok) {
    throw new Error(`Error HTTP: ${respuestaHttp.status}`);
  }

  const datos = await respuestaHttp.json();
  return `📄 ${datos.title}`;
}
```

```
async function enviar(evento) {
  evento.preventDefault();
  const texto = entrada.value.trim();
  if (!texto || btnEnviar.disabled) return;

  pintarMensaje("user", texto);
  entrada.value = "";
  estado.textContent = "Pensando...";
  cambiarEstadoInterfaz(true);

  try {
    const respuesta = await consultarModelo();
    pintarMensaje("ia", respuesta);
    estado.textContent = "✓ Respuesta recibida (fetch OK)";
  } catch (error) {
    estado.textContent = `Error al consultar el modelo: ${error.message}`;
  } finally {
    cambiarEstadoInterfaz(false);
    entrada.focus();
  }
}
```

Se eliminaron `precalentar()`, `setTimeout` y `respuestaPendiente`. `await` garantiza que la respuesta se pinte después de terminar el `fetch`. La variable local evita sobrescrituras y deshabilitar los controles impide solicitudes simultáneas. `finally` recupera la interfaz tanto en éxito como en error.

Procedimiento de limpieza en la pestaña Application

Antes de probar la versión corregida se debe abrir Application > Local Storage, seleccionar el origen, localizar `memoria_1lm` y eliminar la clave con `Delete` o `Supr`; después se recarga la aplicación y se crea un historial nuevo. Este paso es estrictamente necesario porque `JSON.stringify()` solo corrige escrituras futuras: el texto inválido de la versión anterior continúa persistido y `JSON.parse()` volverá a recibir esa basura durante el arranque si no se elimina manualmente.

Comprobación final

- localStorage contiene JSON válido.
- El historial se restaura al recargar.
- Network muestra /posts/{id} con Status 200.
- La respuesta contiene title.
- La interfaz no se congela.
- Las respuestas aparecen después de su pregunta.
- Los errores HTTP se presentan sin dejar bloqueados los controles.

Anexo: evidencia complementaria

Figura 3. Console: JSON.parse falla al recibir [object Object].



Figura 4. Respuestas undefined y solicitudes /postss con estado 404.



Figura 5. Memoria corrupta, respuesta undefined y endpoint incorrecto observados conjuntamente.

